

Construyendo un Data Warehouse

De transacciones OLTP a preguntas OLAP

Diego Villalba

Almacenes y Minería de Datos

Facultad de Ciencias UNAM



**Facultad de
Ciencias**
UNAM

La misión de hoy

Partiremos de **pedidos operacionales** y terminaremos respondiendo:

¿Cuánto vendimos por mes y categoría, sin frenar la operación diaria?

Al final podremos explicar por qué un archivo de clics en JSON abre una pregunta arquitectónica nueva.

Ruta de la hora

Minutos	Lo que haremos	Evidencia
0–10	Preguntas de repaso	Decisiones justificadas
10–22	Definir pregunta, grano y estrella	Diseño en el pizarrón
22–45	Construir juntos	DW ejecutable
45–51	Consultar y validar	Respuesta OLAP
51–58	Romper el molde tabular	Puente a lakehouse
58–60	Cerrar	<i>Exit ticket</i>

Preguntas de repaso

1 · ¿Cuál es la ventaja de OLAP frente a OLTP?

A. OLAP registra una compra individual con menor latencia.

B. OLAP favorece agregaciones históricas y consultas multidimensionales.

C. OLAP elimina la necesidad de integrar fuentes.

D. OLAP garantiza que cualquier consulta sea barata.

Vota primero; justifica después.

1 · ¿Cuál es la ventaja de OLAP frente a OLTP?

A. OLAP registra una compra individual con menor latencia.

B. OLAP favorece agregaciones históricas y consultas multidimensionales.

C. OLAP elimina la necesidad de integrar fuentes.

D. OLAP garantiza que cualquier consulta sea barata.

Respuesta: B. OLAP favorece lecturas amplias, historia, agregaciones y navegación multidimensional.

OLTP y OLAP protegen trabajos distintos

ROW-ORIENTED (OLTP)

Ideal para: **INSERT** , **UPDATE** y lectura de un registro completo.

```
[ 1, Ana, Ventas, 3200 ]  
[ 2, Luis, TI, 4100 ]  
[ 3, Eva, Ventas, 2900 ]  
[ 4, Iván, TI, 4600 ]
```

Para calcular el salario promedio, el motor debe leer **todos** los bloques, aunque solo necesite una columna.

COLUMN-ORIENTED (OLAP)

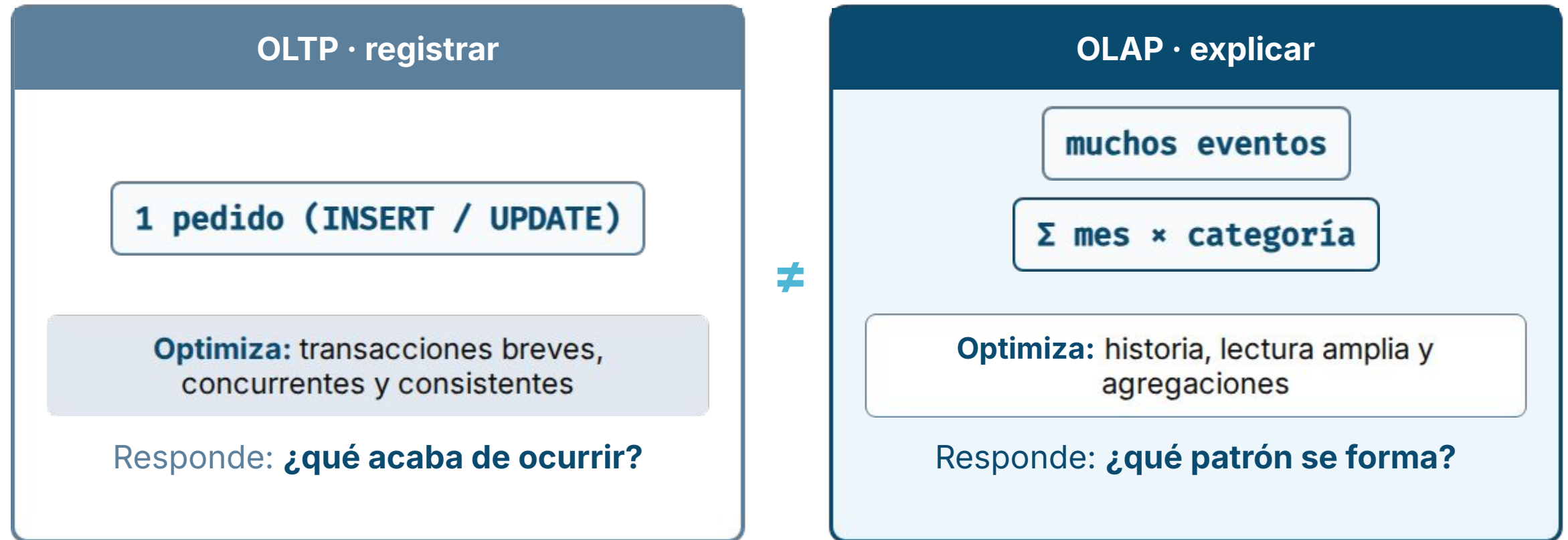
Ideal para: agregaciones (**AVG** , **SUM**) sobre pocas columnas y muchas filas.

```
id: [1, 2, 3, 4]  
nombre: [Ana, Luis, Eva, Iván]  
área: [Ventas, TI, Ventas, TI]  
salario: [3200, 4100, 2900, 4600]
```

Para **AVG(salario)** , el motor lee **únicamente** el bloque de la columna **salario** , ignorando el resto.



OLTP y OLAP protegen trabajos distintos



La ventaja no es "más velocidad" en abstracto: es un diseño adecuado para otra carga de trabajo.

2 · Escalamiento: ¿vertical u horizontal?

↑Vertical · Scale Up

Consiste en añadir CPU/RAM a una sola máquina o servidor principal.

Pro: Simple de implementar.

Contra: Tiene un techo físico y su costo es creciente.

↔Horizontal · Scale Out

Consiste en repartir los datos y el cómputo entre varios nodos en red.

Pro: Es mucho más elástico y escalable.

Contra: Exige partición lógica y coordinación estricta.

2 · Escalamiento: ¿vertical u horizontal?

↑Vertical · Scale Up

Consiste en añadir CPU/RAM a una sola máquina o servidor principal.

Pro: Simple de implementar.

Contra: Tiene un techo físico y su costo es creciente.



UN SERVIDOR
+ CPU · + RAM

Simple; llega a un techo.

↔Horizontal · Scale Out

Consiste en repartir los datos y el cómputo entre varios nodos en red.

Pro: Es mucho más elástico y escalable.

Contra: Exige partición lógica y coordinación estricta.



nodo 1

nodo 2

nodo 3

Elástico; exige partición, red y coordinación.



1 · ¿Cuál es la ventaja de OLAP frente a OLTP?

El CRM corrige la ciudad de una clienta y el reporte del año pasado cambia sin dejar rastro.

¿Qué propiedad clásica del Data Warehouse quedó comprometida?

A. Orientado a temas

B. Integrado

C. Variante en el tiempo

D. No volátil

Vota primero; justifica después.

1 · ¿Cuál es la ventaja de OLAP frente a OLTP?

El CRM corrige la ciudad de una clienta y el reporte del año pasado cambia sin dejar rastro.

¿Qué propiedad clásica del Data Warehouse quedó comprometida?

A. Orientado a temas

B. Integrado

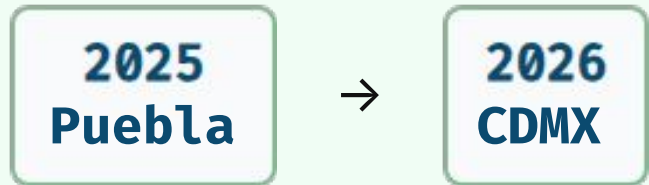
C. Variante en el tiempo

D. No volátil

Vota primero; justifica después.

3 · ¿Qué propiedad se pierde al sobrescribir?

✓ DW con Historia



Mantenemos el estado de 2025 (Puebla) y el nuevo de 2026 (CDMX).

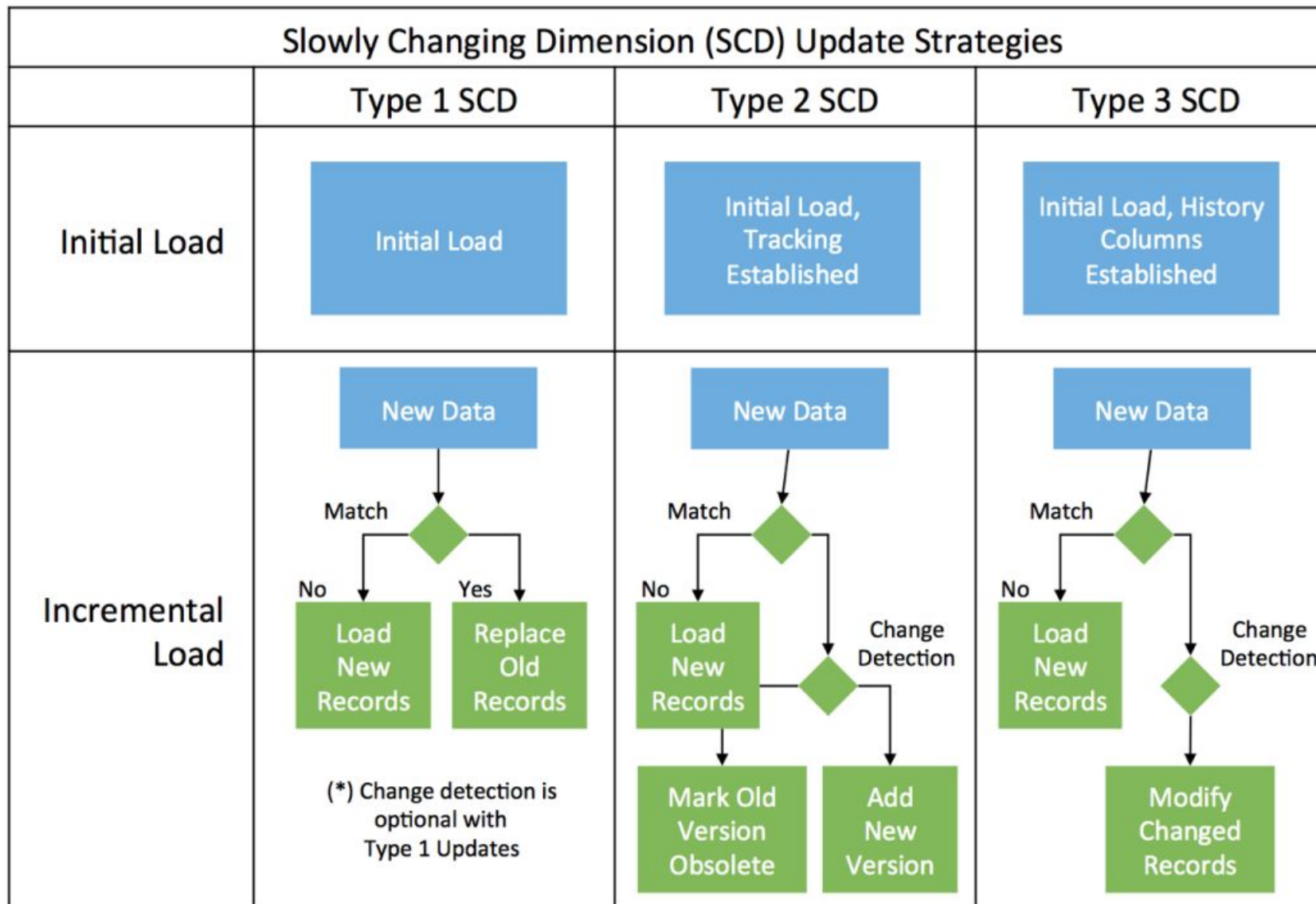
Ambos estados permanecen, garantizando la trazabilidad de los reportes pasados.

✗ Sobrescribir



Remplazamos el dato de Puebla directamente por CDMX en el registro.

El reporte del año pasado cambia sin dejar rastro. Se pierde la propiedad "Variante en el tiempo".



Diseñemos antes de programar

Caso · Café Ciencia

Tenemos cuatro tablas operacionales:

clientes · productos · pedidos · detalle_pedido

Dirección pregunta:

*¿Cómo evoluciona la **venta neta** por mes, categoría y ciudad?*

Criterio de éxito: una sola consulta reproducible, con totales conciliados contra la fuente.

La decisión que manda sobre el modelo

¿Cuál será el grano de la tabla de hechos?

- Una fila por cliente
- Una fila por pedido
- Una fila por renglón de pedido completado
- Una fila por mes

La decisión que manda sobre el modelo

¿Cuál será el grano de la tabla de hechos?

- Una fila por cliente
- Una fila por pedido
- **Una fila por renglón de pedido completado**
- Una fila por mes

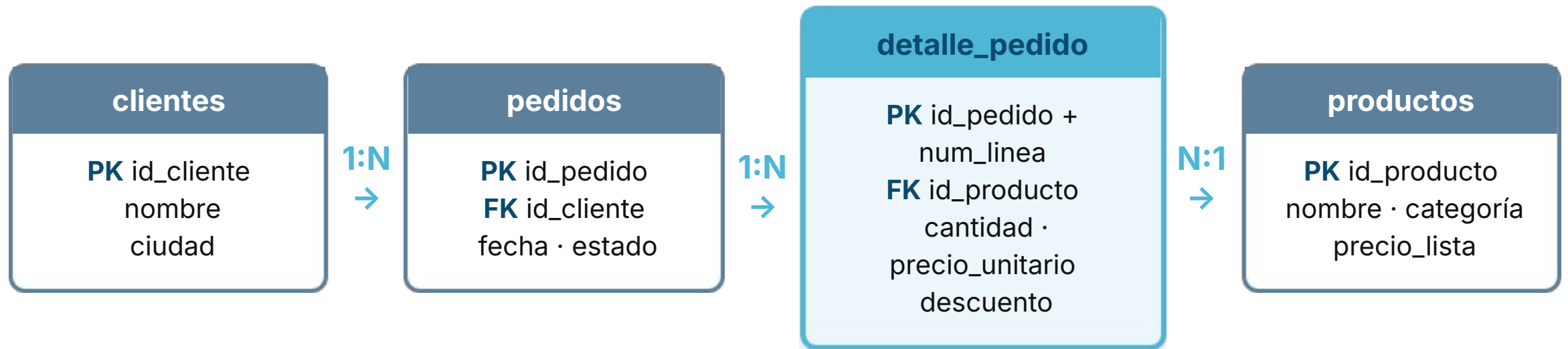
El grano se declara **antes** de elegir dimensiones o métricas.

Elección: un renglón de pedido completado.

Con el renglón de pedido podemos:

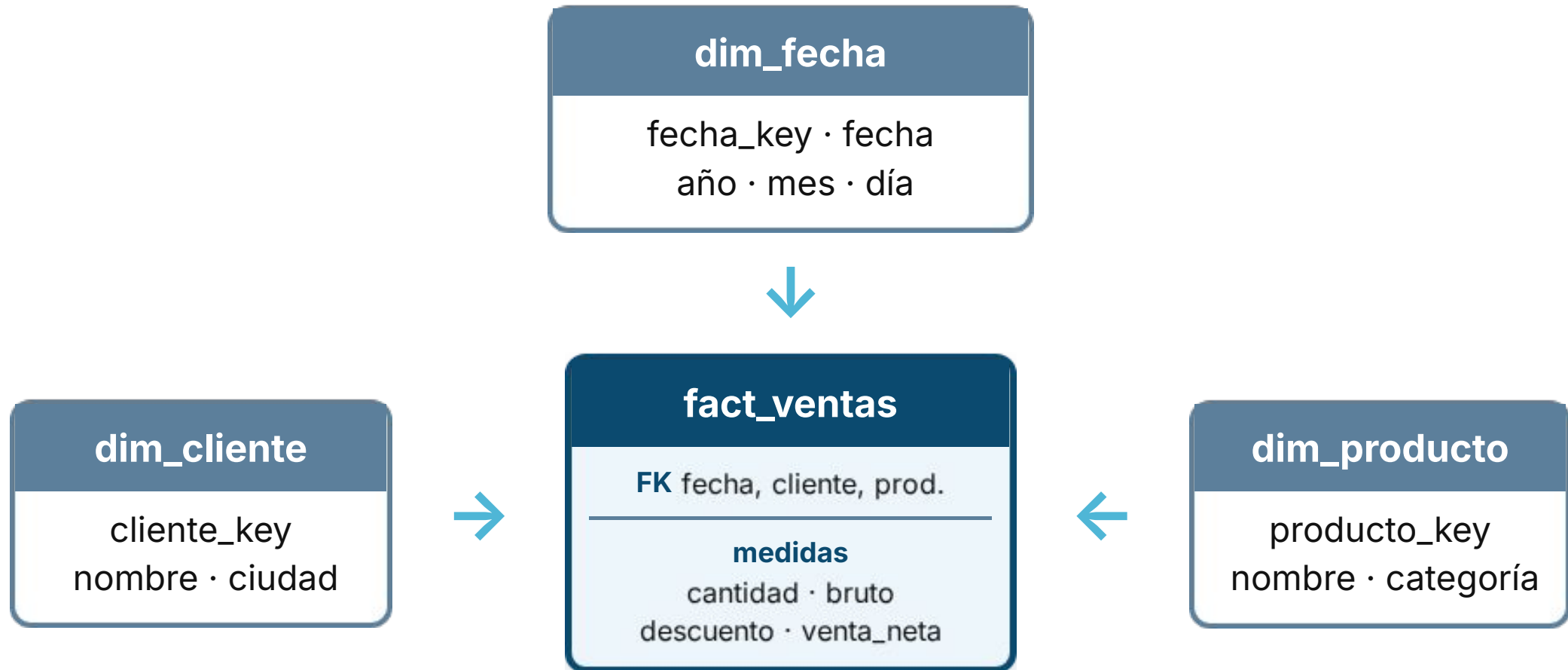
- sumar cantidades e importes;
- analizar producto y categoría;
- reagrupar después por pedido, día o mes.

Fuente OLTP · buena para escribir



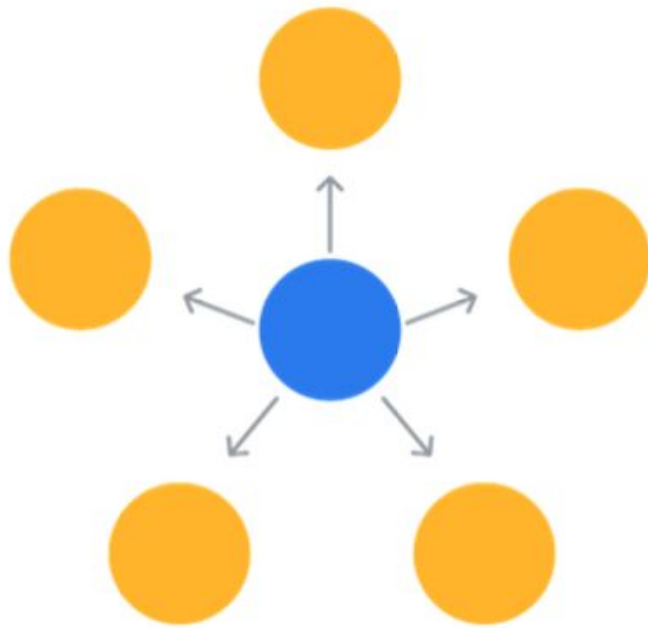
- Evita repetir nombres y categorías en cada transacción.
- Protege integridad durante muchas escrituras concurrentes.
- Obliga a varios *joins* para preguntas históricas.

Destino OLAP · esquema estrella



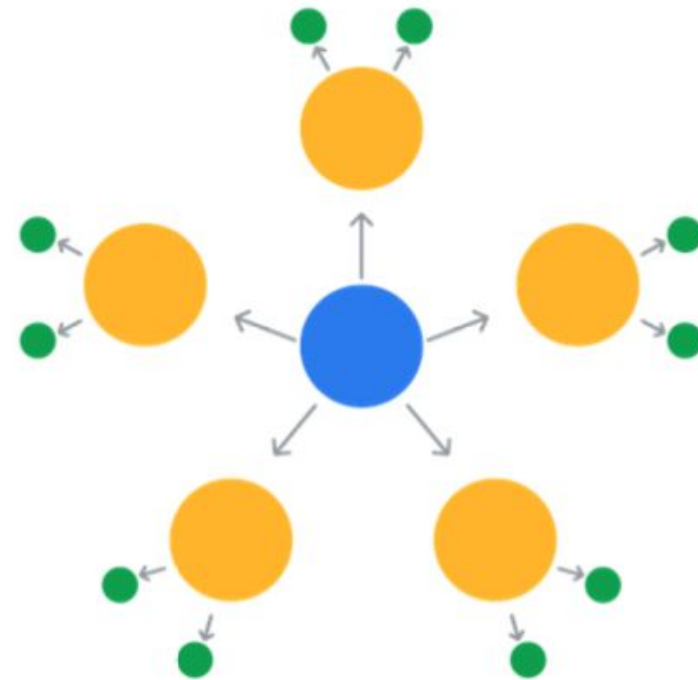
Destino OLAP · esquema estrella

Star schema



vs

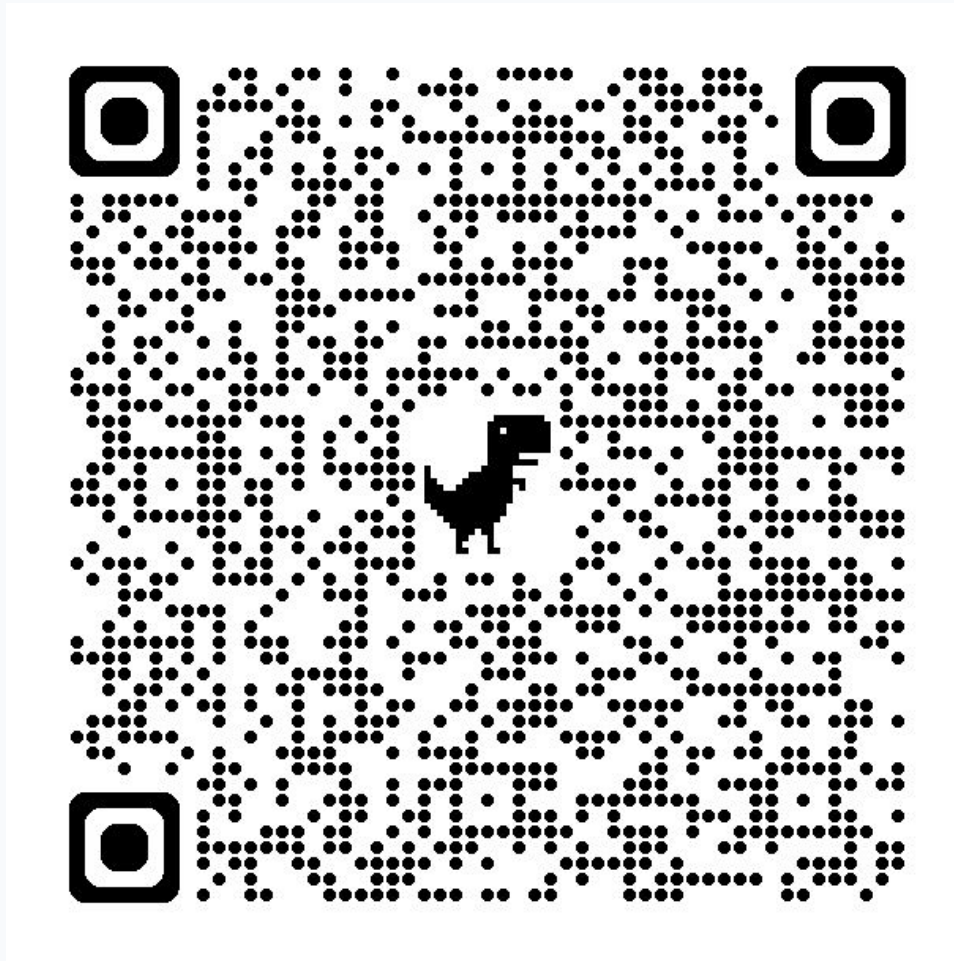
Snowflake schema



Destino OLAP · esquema estrella

Decisión	Elección de hoy
Hecho	Renglón de pedido completado
Dimensiones	Fecha, cliente, producto
Medidas aditivas	Cantidad, bruto, descuento, neto
Claves del DW	Sustitutas, independientes del origen

Construyamos juntos



Dos modos, el mismo aprendizaje



Modo demostración

1. El profesor ejecuta.
2. El grupo predice cada salida.
3. El diseño se reconstruye en el pizarrón.



Modo Colab

1. Abre el laboratorio en Colab.
2. **Entorno de ejecución → Ejecutar todas.**
3. Compara validaciones con la pantalla.

`laboratorio-data-warehouse.ipynb`

`.env`

Sin internet: abra localmente con el entorno del repositorio.



Facultad de
Ciencias
UNAM

Paso 1 · Crear las dimensiones

```
CREATE TABLE dim_cliente (  
  cliente_key INTEGER PRIMARY KEY,  
  cliente_id_origen INTEGER,  
  nombre TEXT,  
  ciudad TEXT  
);  
  
CREATE TABLE dim_producto (  
  producto_key INTEGER PRIMARY KEY,  
  producto_id_origen INTEGER,  
  nombre TEXT, categoria TEXT  
);
```

Pregunta antes de ejecutar: ¿por qué guardamos la clave de origen y otra clave propia?

Salida

- ✓ dim_fecha
- ✓ dim_cliente
- ✓ dim_producto

3 dimensiones creadas



Paso 2 · Crear la tabla de hechos

```
CREATE TABLE fact_ventas (  
  pedido_id_origen INTEGER,  
  num_linea INTEGER,  
  fecha_key INTEGER,  
  cliente_key INTEGER,  
  producto_key INTEGER,  
  cantidad INTEGER,  
  importe_bruto REAL,  
  descuento REAL,  
  venta_neta REAL,  
  PRIMARY KEY (pedido_id_origen, num_linea)  
);
```

La llave primaria protege el **grano** declarado.

Salida

- ✓ creada **fact_ventas**
- ✓ 3 claves foráneas
- ✓ 4 medidas

PK (pedido, línea)

Paso 3 · Extraer y filtrar

```
filas = oltp.execute("""
SELECT p.id_pedido, d.num_linea,
       p.fecha, p.id_cliente,
       d.id_producto, d.cantidad,
       d.precio_unitario, d.descuento
FROM pedidos p
JOIN detalle_pedido d USING (id_pedido)
WHERE p.estado = 'COMPLETADO'
""").fetchall()
```

Regla de negocio: un pedido cancelado no es venta.

Predicción: ¿cuántos renglones pasarán a la tabla de hechos?

Resultado

7 renglones pasan

2 quedan fuera
(pedido 1003 cancelado)



Paso 4 · Transformar y cargar

```
for f in filas:
    bruto = round(f[5] * f[6], 2)
    neto = round(bruto - f[7], 2)

    dw.execute(
        "INSERT INTO fact_ventas VALUES "
        "(?, ?, ?, ?, ?, ?, ?, ?, ?)",
        (f[0], f[1], fecha_key(f[2]),
         cliente_keys[f[3]],
         producto_keys[f[4]],
         f[5], bruto, f[7], neto)
    )
```

ETL: extraer → aplicar reglas y claves → cargar.

Primera fila cargada

pedido 1001 · línea 1

cantidad

bruto

descuento

venta_neta

✓ 7 filas cargadas



Facultad de
Ciencias
UNAM

Paso 5 · No confiar: validar

```
assert filas_fuente == filas_hecho == 7 assert  
round(neto_fuente, 2) == round(neto_dw, 2) ==  
702.50 assert claves_huerfanas == 0
```

Una carga que "terminó sin error" todavía puede estar equivocada.

SALIDA DE VALIDACIÓN

Contrato	Resultado
✓ Renglones	7
✓ Venta neta	\$702.50
✓ Huérfanas	0

Consultemos como analistas

Consulta OLAP · *roll-up*

```
SELECT f.anio, f.mes, p.categoria,  
SUM(v.cantidad) AS unidades,  
ROUND(SUM(v.venta_neta), 2) AS  
venta_neta FROM fact_ventas v JOIN  
dim_fecha f USING (fecha_key) JOIN  
dim_producto p USING (producto_key)  
GROUP BY f.anio, f.mes, p.categoria  
ORDER BY f.anio, f.mes, p.categoria;
```

Resultado de la consulta

Año	Mes	Categoría	Unidades	Neto
2026	1	Bebidas	4	\$317.50
2026	1	Equipo	1	\$120.00
2026	2	Alimentos	4	\$205.00
2026	2	Bebidas	1	\$60.00

¿Qué compramos con el Data Warehouse?

Antes

- Modelo para registrar pedidos
- Estado operativo mutable
- Reglas repartidas entre tablas
- Consultas que compiten con la operación

Después

- Grano analítico explícito
- Historia lista para comparar
- Dimensiones con lenguaje de negocio
- Métricas conciliadas y reutilizables

Costo: duplicación controlada, ETL, pruebas, gobierno y mantenimiento.

Cuando una fila ya no basta

Llega el comportamiento digital

```
{ "evento": "ver_producto", "ts":  
"2026-02-03T", "cliente_id": 2, "producto": { "id":  
103, "categoria": "Alimentos" }, "dispositivo":  
{"tipo": "movil"} }
```

```
// Nuevo evento añade estructura no planificada {  
"experimento": { "nombre": "checkout_v2",  
"variante": "B" } }
```

El evento siguiente agrega una estructura que antes no existía:

Llaves observadas:

- Evento 1: **cliente_id** **dispositivo**
evento **producto** **ts**
- Evento 2: las anteriores + **experimento**

Y además llegan reseñas en texto e imágenes de productos.

Pregunta: ¿qué columnas crearíamos sin perder el original?

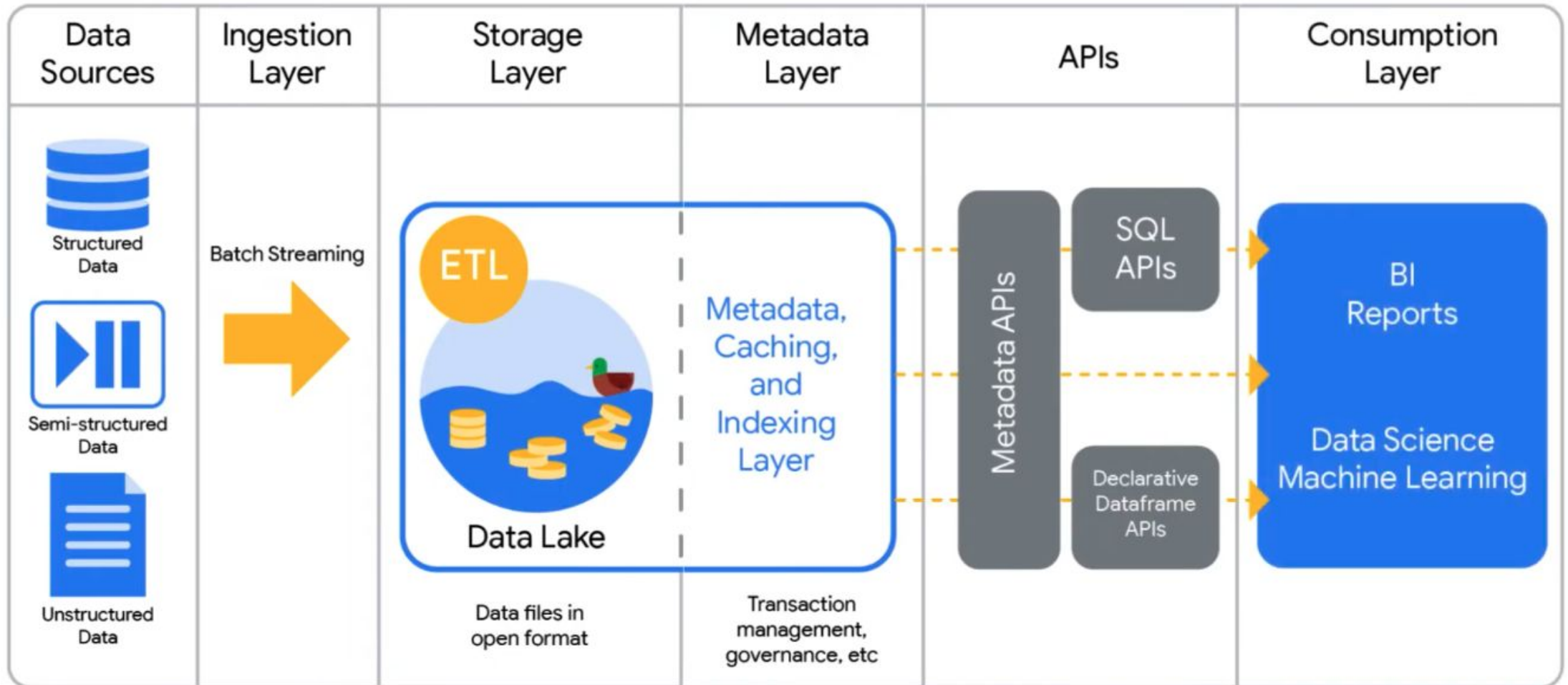
No es "JSON $\Rightarrow$ lakehouse"

Un Data Warehouse moderno puede almacenar JSON, y un proceso ETL puede tabularlo.

La tensión aparece cuando necesitamos simultáneamente:

- conservar datos crudos diversos a gran escala;
- aceptar evolución de esquema sin perder evidencia;
- servir BI **y** ciencia de datos / ML;
- separar y escalar almacenamiento y cómputo;
- administrar archivos como tablas confiables.

Puente a Data Lakehouse



Puente a Data Lakehouse



Parquet organiza un archivo; Delta, Iceberg o Hudi coordinan tablas formadas por muchos archivos.

Cierre

Exit ticket · 60 segundos

En una frase por punto:

1. ¿Por qué no consultamos directamente el OLTP para todo?
2. ¿Cuál fue el grano de y qué protegió? `fact_ventas`
3. ¿Qué requisito —no qué formato— podría llevarnos a un lakehouse?
- .

Próxima clase

De archivos diversos a **tablas abiertas, gobernadas y versionadas**.